

Code-Layout, Readability and Reusability

Date	03 July 2026
Team ID	SWTID-2026-1614
Project Name	A Comprehensive Measure of Well-Being
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	4-space indentation is consistent in Flask/app.py and Training/train_model.py. HTML templates are cleanly nested.
2	Proper File Structure	Files and folders are logically organized	Yes	Separation of backend serving (Flask/), training modules (Training/), and dataset tables (Dataset/).
3	Meaningful Variable Names	Variables reflect their purpose clearly	Partial	life_expectancy, schooling, and gni_capita are descriptive. Helper arrays use standard short variable labels.
4	Function / Method Names	Functions are descriptively named	Yes	home() and predict_hdi() clearly define their routes and logic roles.
5	Code Comments	Inline and block comments explain logic	Yes	Inline comments detail pandas data imports, sklearn regression initialization, and server configuration.
6	Modular Design	Code is split into reusable functions/modules	Partial	Data training and data serving are in distinct folders, but API routing and inference are coupled in app.py.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Unused templates and backup scripts have been removed to keep the directory layout clean.
8	Error Handling	Exceptions and errors are handled gracefully	Partial	Basic validation ensures fields are populated, though nested error bounds (e.g. string inputs on numbers) can be improved.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High/Medium/Low)
1	hdi_model.pkl (trained model)	Python / scikit-learn	Loaded in Flask/app.py at launch to handle all simulation requests.	High
2	HDI.csv (indicators dataset)	CSV Flat-File	Read in Flask/app.py to compile the country name autocompletion lookup list.	High
3	indexnew.html form template	HTML5 / CSS3 / JS	Renders the primary user interface and binds client forms.	Medium
4	Training/train_model.py	Python / Pandas	Executed independently to train weights when fresh annual metrics are available.	High
5	predict_hdi() route logic	Python / Flask	Embedded service logic; tied directly to Flask endpoint mappings.	Low

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clean directory mapping, ensuring separate folders for training, dataset, and backend servers. Consistent tab spaces.
Readability	5	Key socio-economic variables are fully written out (e.g. mean_years_schooling), making code review easy.
Reusability	5	The ML training scripts are isolated from client serving, meaning models can be retrained without affecting interface files.
Documentation / Comments	5	Exhaustive README.md mapping out execution paths, project directories, and execution steps.
Overall Score	5	A clean, modular, and well-commented repository. Future improvements include adding strict client-side validation logic.